

如何在 Flask 應用程式使用 App Engine 工作佇列 (推送工作) (模組 7)

來源：<https://codelabs.developers.google.com/codelabs/cloud-gae-python-migrate-7-gaetasks?hl=zh-tw>

步驟數：8

瞭解如何將工作佇列推送工作使用情形新增至基本的 Python 2 Flask App Engine NDB 應用程式。

目錄

1. [1. 總覽](#)
2. [2. 背景](#)
3. [3. 設定/準備工作](#)
4. [4. 更新設定](#)
5. [5. 修改應用程式檔案](#)
6. [6. 摘要/清除](#)
7. [7. 遷移至 Python 3](#)
8. [8. 其他資源](#)

1. 總覽

[無伺服器遷移工作站系列程式碼研究室](#) (自學式實作教學課程) 和[相關影片](#)，旨在協助 [Google Cloud 無伺服器](#)開發人員完成一或多項遷移作業 (主要是從舊版服務遷移)，進而翻新應用程式。這樣做可提高應用程式的可攜性，並提供更多選項和彈性，讓您整合及存取更多 Cloud 產品，並更輕鬆地升級至新版語言。雖然一開始的重點是早期 Cloud 使用者，主要是 [App Engine](#) (標準環境) 開發人員，但本系列涵蓋範圍廣泛，也包括其他無伺服器平台，例如 [Cloud Functions](#) 和 [Cloud Run](#)，或適用於其他平台。

本程式碼研究室會說明如何在 [第 1 模組程式碼研究室](#)的範例應用程式中使用 [App Engine 工作佇列推送工作](#)。單元 7 的[網誌文章](#)和[影片](#)是本教學課程的補充資料，簡要介紹本教學課程的內容。

在本單元中，我們會新增[推送工作](#)的使用方式，然後在第 8 單元將該使用方式遷移至 Cloud Tasks，並在第 9 單元遷移至 Python 3 和 Cloud Datastore。使用工作佇列處理[提取工作](#)的開發人員，將遷移至 Cloud Pub/Sub，並應改為參閱第 18 至 19 節。

未使用工作佇列？

如果您的應用程式未使用 App Engine 工作佇列服務，請略過第 7 到 9 節和第 18 到 19 節，或將這些程式碼研究室當做練習，熟悉工作佇列遷移作業。

在接下來的研究室中

- 使用 App Engine 工作佇列 API/套裝組合服務
- 在基本 Python 2 Flask App Engine NDB 應用程式中新增推送工作用量

軟硬體需求

- 擁有[Google Cloud 專案](#)，且已啟用 [GCP 帳單帳戶](#)
- Python 基礎技能
- 熟悉常見的 Linux 指令
- 具備[開發](#)及[部署](#) App Engine 應用程式的基本知識
- 可正常運作的第 1 模組 App Engine 應用程式 (建議完成[程式碼研究室](#)，或從存放區複製[應用程式](#))

問卷調查

2. 背景

[App Engine 工作佇列](#)支援推送和提取工作。為提升應用程式可攜性，Google Cloud 團隊建議您從 Task Queue 等舊版套裝組合服務，遷移至其他 Cloud 獨立或第三方同等服務。

- [工作佇列發送工作](#)使用者應遷移至 [Cloud Tasks](#)。
- [工作佇列提取工作](#)的使用者應遷移至 [Cloud Pub/Sub](#)。

遷移單元 18-19 涵蓋提取工作遷移，單元 7-9 則著重於推送工作遷移。如要從 App Engine Task Queue 推送工作遷移，請將其用量新增至「[第 1 堂程式碼研究室](#)」產生的現有 Flask 和 App Engine NDB 應用程式。在該應用程式中，新的網頁瀏覽會註冊新的造訪，並向使用者顯示最近的造訪記錄。由於系統不會再顯示舊的造訪記錄，且這些記錄會佔用 Datastore 的空間，因此我們要建立推送工作，自動刪除最舊的造訪記錄。在第 8 個單元中，我們會將該應用程式從工作佇列遷移至 Cloud Tasks。

本教學課程包含下列步驟：

1. 設定/準備工作
 2. 更新設定
 3. 修改應用程式程式碼
-

3. 設定/準備工作

本節將說明如何：

1. 設定 Cloud 專案
2. 取得基準範例應用程式
3. (重新) 部署及驗證基準應用程式

這些步驟可確保您一開始使用的是可運作的程式碼。

1. 設定專案

如果您已完成[第 1 模組程式碼研究室](#)，建議重複使用該專案 (和程式碼)。或者，您也可以[建立全新專案](#)，或重複使用其他現有專案。確認專案已啟用 App Engine，且具備有效的帳單帳戶。

2. 取得基準範例應用程式

本程式碼研究室的先決條件之一，是必須有可運作的第 1 模組 App Engine 應用程式：完成[第 1 模組程式碼研究室](#) (建議)，或從存放區複製[第 1 模組應用程式](#)。無論您使用自己的程式碼或我們的程式碼，模組 1 的程式碼都是「起點」。本程式碼研究室會逐步說明每個步驟，最後的程式碼會與第 7 模組存放區資料夾「FINISH」中的程式碼類似。

- 開始：[模組 1 資料夾](#) (Python 2)
- 完成：[模組 7 資料夾](#) (Python 2)
- [整個存放區](#) (複製或下載 [ZIP 檔案](#))

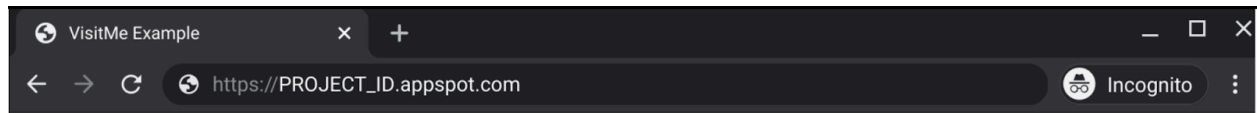
無論您使用哪個第 1 模組應用程式，資料夾都應如下所示，可能也會有 `lib` 資料夾：

```
$ ls
README.md      main.py        templates
app.yaml      requirements.txt
```

3. (重新) 部署基準應用程式

請執行下列步驟，(重新) 部署第 1 課的應用程式：

1. 刪除 `lib` 資料夾 (如有)，然後執行 `pip install -t lib -r requirements.txt` 重新填入 `lib`。如果您同時安裝了 Python 2 和 3，可能需要改用 `pip2` 指令。
2. 請確認您已[安裝及初始化](#) `gcloud` 指令列工具，並已瞭解其用法。
3. 如不想在發出的每個 `gcloud` 指令中輸入 `PROJECT_ID`，請使用 `gcloud config set project PROJECT_ID` 設定 Cloud 專案。
4. 使用 `gcloud app deploy` 部署範例應用程式
5. 確認模組 1 應用程式可正常運作，且會顯示最近的造訪記錄 (如下圖所示)



VisitMe example

Last 10 visits

- Tue Mar 30 06:49:20 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (Macintosh; Intel Mac OS X 11_2_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.90 Safari/537.36
 - Sun Mar 28 03:45:17 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (X11; CrOS x86_64 13597.94.0) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/88.0.4324.186 Safari/537.36
 - Sat Mar 27 20:31:55 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: curl/7.64.0
 - Sat Mar 27 20:23:39 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (X11; CrOS x86_64 13597.94.0) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/88.0.4324.186 Safari/537.36
 - Fri Mar 26 04:09:23 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: curl/7.64.1
 - Fri Mar 26 04:07:26 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: curl/7.64.1
 - Sat Mar 20 00:17:54 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: curl/7.64.1
 - Sat Mar 20 00:02:06 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (Macintosh; Intel Mac OS X 11_2_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.82 Safari/537.36
 - Sat Mar 20 00:00:15 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (Macintosh; Intel Mac OS X 11_2_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.82 Safari/537.36
 - Fri Mar 19 23:28:54 2021 from 127.0.0.1: Mozilla/5.0 (Macintosh; Intel Mac OS X 11_2_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.82 Safari/537.36
-

步驟 4 · 約 1 分鐘

4. 更新設定

標準 App Engine 設定檔 (`app.yaml` 、 `requirements.txt` 、 `appengine_config.py`) 無須變更。

5. 修改應用程式檔案

主要應用程式檔案為 `main.py`，本節中的所有更新都與該檔案有關。此外，網頁範本 `templates/index.html` 也進行了小幅更新。本節要實作的變更如下：

1. 更新匯入作業
2. 新增推送工作
3. 新增工作處理常式
4. 更新網頁範本

1. 更新匯入作業

匯入 `google.appengine.api.taskqueue` 會引入 Task Queue 功能。此外，您也需要部分 Python 標準程式庫套件：

- 由於我們要新增刪除最舊造訪記錄的工作，應用程式必須處理時間戳記，也就是使用 `time` 和 `datetime`。
- 如要記錄與工作執行相關的實用資訊，我們需要 `logging`。

新增所有匯入內容後，程式碼在變更前後的樣子如下所示：

BEFORE:

```
from flask import Flask, render_template, request
from google.appengine.ext import ndb
```

修改後：

```
from datetime import datetime
import logging
import time
from flask import Flask, render_template, request
from google.appengine.api import taskqueue
from google.appengine.ext import ndb
```

2. 新增推送工作 (彙整工作資料、將新工作加入佇列)

發送佇列說明文件指出：「如要處理工作，您必須將其新增至發送佇列。App Engine 提供一個名為 `default` 的預設發送佇列，這個佇列已設定好，隨時可以配合預設設定一起運用。您可以視需要將所有工作新增至預設佇列，不需建立及設定其他佇列。為簡化程式碼，本程式碼研究室使用 `default` 佇列。如要進一步瞭解如何定義具有相同或不同特徵的自有推送佇列，請參閱「[建立推送佇列](#)」說明文件。

本程式碼研究室的主要目標是新增工作 (至 `default` 推送佇列)，這項工作會從 Datastore 刪除不再顯示的舊版造訪記錄。基線應用程式會建立新的 `Visit` 實體，藉此註冊每次造訪 (`GET` 要求至 `/`)，然後擷取並顯示最近的造訪記錄。系統不會再顯示或使用任何最舊的造訪記錄，因此推送工作會刪除所有比顯示的最舊記錄更舊的造訪記錄。為達成上述目的，應用程式的行為需要稍做變更：

1. 查詢最近的造訪記錄時，請修改應用程式，儲存最後一個 `Visit` 的時間戳記 (顯示時間最舊的造訪記錄)，而非立即傳回這些記錄。您可以安全地刪除所有比這個時間戳記更舊的造訪記錄。
2. 以這個時間戳記做為酬載建立發送工作，並將工作導向工作處理常式，該常式可透過 HTTP `POST` 存取 `/trim`。具體來說，請使用標準 Python 公用程式轉換 Datastore 時間戳記，並將其 (以浮點數形式) 傳送至工作，同時記錄該時間戳記 (以字串形式)，然後將該字串做為信號值傳回，供使用者查看。

所有這些作業都會在 `fetch_visits()` 中進行，更新前後的樣子如下：

BEFORE:

```
def fetch_visits(limit):
    return (v.to_dict() for v in Visit.query().order(
        -Visit.timestamp).fetch(limit))
```

修改後：

```
def fetch_visits(limit):
    'get most recent visits and add task to delete older visits'
    data = Visit.query().order(-Visit.timestamp).fetch(limit)
    oldest = time.mktime(data[-1].timestamp.timetuple())
    oldest_str = time.ctime(oldest)
    logging.info('Delete entities older than %s' % oldest_str)
    taskqueue.add(url='/trim', params={'oldest': oldest})
    return (v.to_dict() for v in data), oldest_str
```

3. 新增工作處理常式 (工作執行時呼叫的程式碼)

雖然在 `fetch_visits()` 中刪除舊訪次很容易，但請注意，這項功能與使用者關係不大。這項輔助功能很適合在標準應用程式要求以外，以非同步方式處理。Datastore 中的資訊較少，因此查詢速度更快，使用者可從中獲益。建立新的函式 `trim()`，透過傳送至 `/trim` 的 Task Queue POST 要求呼叫，並執行下列動作：

1. 擷取「最舊的造訪記錄」時間戳記酬載
2. 發出 Datastore 查詢，找出早於該時間戳記的所有實體。
3. 由於不需要實際使用者資料，因此選擇速度較快的「僅限鍵」查詢。
4. 記錄要刪除的實體數量 (包括零)。
5. 呼叫 `ndb.delete_multi()` 刪除任何實體 (如果沒有則略過)。
6. 傳回空字串 (以及隱含的 HTTP 200 傳回代碼)。

詳情請參閱下方的 `trim()`。在 `fetch_visits()` 後方，將此程式碼新增至 `main.py`：

```
@app.route('/trim', methods=['POST'])
def trim():
    '(push) task queue handler to delete oldest visits'
    oldest = request.form.get('oldest', type=float)
    keys = Visit.query(
        Visit.timestamp < datetime.fromtimestamp(oldest)
    ).fetch(keys_only=True)
    nkeys = len(keys)
    if nkeys:
        logging.info('Deleting %d entities: %s' % (
            nkeys, ', '.join(str(k.id()) for k in keys)))
        ndb.delete_multi(keys)
    else:
        logging.info('No entities older than: %s' % time.ctime(oldest))
    return '' # need to return SOME string w/200
```

4. 更新網頁範本

使用這個 Jinja2 條件更新網頁範本 `templates/index.html`，如果該變數存在，則顯示最舊的時間戳記：

```
{% if oldest is defined %}
    <b>Deleting visits older than:</b> {{ oldest }}</p>
{% endif %}
```

在顯示的造訪清單後，但在關閉主體前新增這個程式碼片段，範本看起來會像這樣：

```
<!doctype html>
<html>
<head>
<title>VisitMe Example</title>
<body>

<h1>VisitMe example</h1>
<h3>Last 10 visits</h3>
<ul>
{% for visit in visits %}
    <li>{{ visit.timestamp.ctime() }} from {{ visit.visitor }}</li>
{% endfor %}
</ul>

{% if oldest is defined %}
    <b>Deleting visits older than:</b> {{ oldest }}</p>
{% endif %}
</body>
</html>
```

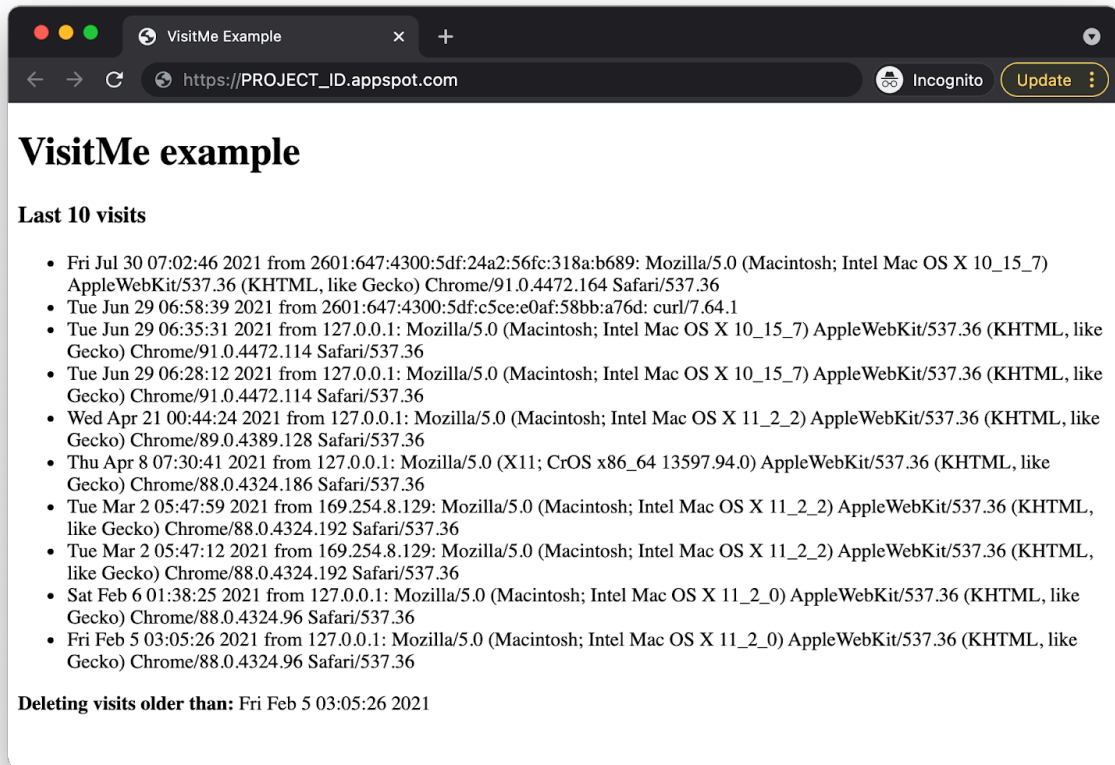
步驟 6 · 約 4 分鐘

6. 摘要/清除

本節將部署應用程式，並驗證應用程式是否正常運作，以及是否反映在輸出內容中，為本程式碼研究室畫下句點。應用程式驗證完成後，請執行任何清理作業，並考慮後續步驟。

部署及驗證應用程式

使用 `gcloud app deploy` 部署應用程式。輸出內容應與第 1 模組的應用程式相同，但底部會顯示新的一行，說明要刪除哪些造訪記錄：



恭喜您完成本程式碼研究室。現在您的程式碼應該與 [Module 7 repo](#) 資料夾中的程式碼相符。現在可以遷移至第 8 模組中的 [Cloud Tasks](#)。

清除所用資源

一般

如果暫時不需要使用，建議[停用 App Engine 應用程式](#)，以免產生費用。不過，如果您想進一步測試或實驗，App Engine 平台提供[免付費配額](#)，只要不超出該用量層級，就不會產生費用。這是指運算費用，但您可能也需要支付相關 App Engine 服務的費用，因此請參閱[其定價頁面](#)瞭解詳情。如果這項遷移作業涉及其他雲端服務，則這些服務會另外計費。無論是哪種情況，請視需要參閱下方的「本程式碼研究室專用」一節。

為求完整揭露，部署至 App Engine 等 Google Cloud 無伺服器運算平台時，會產生[少量建構和儲存空間費用](#)。[Cloud Build](#) 和 [Cloud Storage](#) 都有各自的免費配額。儲存該圖片會耗用部分配額。不過，你所在的區域可能沒有這類免付費層級，因此請留意儲存空間用量，盡量減少潛在費用。您應審查的特定 Cloud Storage「資料夾」包括：

- `console.cloud.google.com/storage/browser/LOC.artifacts.PROJECT_ID.appspot.com/containers/images`

- `console.cloud.google.com/storage/browser/staging.PROJECT_ID.appspot.com`
- 上述儲存空間連結取決於您的 `PROJECT_ID` 和 * `LOCATION`，例如，如果您的應用程式託管於美國，則為「`us`」。

另一方面，如果您不打算繼續使用這個應用程式或其他相關的遷移 Codelab，並想完全刪除所有內容，請[關閉專案](#)。

本程式碼研究室專用

下列服務是本程式碼研究室的專屬服務。詳情請參閱各項產品的說明文件：

- 根據[舊版套裝服務的定價頁面](#)，App Engine 工作佇列服務不會產生任何額外費用，例如工作佇列。
- [App Engine Datastore](#) 服務是由 [Cloud Datastore](#) (Cloud Firestore Datastore 模式) 提供，這項服務也有免費方案；詳情請參閱[定價頁面](#)。

後續步驟

在這次「遷移」中，您已將 Task Queue 發送佇列用法新增至第 1 課的範例應用程式，並新增追蹤訪客的支援功能，因此產生第 7 課的範例應用程式。下一次遷移會說明如何從 App Engine 發送工作升級至 Cloud Tasks (如果選擇這麼做)。自 2021 年秋季起，使用者升級至 Python 3 時，不再需要遷移至 Cloud Tasks。詳情請參閱下一節。

如果您想遷移至 Cloud Tasks，請繼續進行[第 8 堂程式碼研究室](#)。此外，您還需要考慮其他遷移作業，例如 Cloud Datastore、Cloud Memorystore、Cloud Storage 或 Cloud Pub/Sub (提取佇列)。此外，您也可以將其他產品遷移至 Cloud Run 和 Cloud Functions。所有 [Serverless Migration Station 內容](#) (程式碼研究室、影片、原始碼 [如有]) 都可在[開放原始碼存放區](#)存取。

7. 遷移至 Python 3

2021 年秋季，App Engine 團隊將許多隨附服務的支援範圍擴大至第 2 代執行階段 (原本僅適用於第 1 代執行階段)，這表示將應用程式移植到 Python 3 時，您不必再從 App Engine Task Queue 等隨附服務遷移至獨立的 Cloud 或第三方對等服務 (例如 Cloud Tasks)。換句話說，只要您改造程式碼，[從新一代執行階段存取套裝組合服務](#)，就能繼續在 Python 3 App Engine 應用程式中使用工作佇列。

如要進一步瞭解如何將套裝服務的使用情形遷移至 Python 3，請參閱[第 17 堂程式碼研究室](#)和相關影片。雖然這個主題超出第 7 堂課程的範圍，但下方連結提供第 1 堂和第 7 堂課程的應用程式 Python 3 版本，這些應用程式已移植到 Python 3，但仍使用 App Engine NDB 和工作佇列。

8. 其他資源

以下列出其他資源，供開發人員進一步瞭解這個或相關的遷移模組，以及相關產品。包括提供內容意見回饋的位置、程式碼連結，以及您可能會覺得實用的各種文件。

程式碼研究室問題/意見回饋

如果發現本程式碼研究室有任何問題，請先搜尋問題，再提出回報。搜尋及建立新問題的連結：

- [搜尋問題](#)
- [回報新問題/提供意見](#)

遷移資源

下表提供第 2 堂課 (開始) 和第 7 堂課 (完成) 的存放區資料夾連結。

Codelab	Python 2	Python 3
Module 1	code	程式碼 (本教學課程未介紹)
單元 7 (本程式碼研究室)	code	程式碼 (本教學課程未介紹)

線上資源

以下是可能與本教學課程相關的線上資源：

App Engine Task Queue

- [App Engine 工作佇列總覽](#)
- [App Engine Task Queue 推送佇列總覽](#)
- [建立 Task Queue 發送佇列](#)
- [queue.yaml 參考資料](#)
- [queue.yaml 與 Cloud Tasks 比較](#)
- [發送佇列至 Cloud Tasks 遷移指南](#)
- [App Engine 工作佇列發送佇列至 Cloud Tasks 說明文件範例](#)

App Engine 平台

- [App Engine 說明文件](#)
- [Python 2 App Engine \(標準環境\) 執行階段](#)
- [在 Python 2 App Engine 上使用 App Engine 內建程式庫](#)
- [Python 3 App Engine \(標準環境\) 執行階段](#)
- [Python 2 和 3 App Engine \(標準環境\) 執行階段的差異](#)
- [Python 2 到 3 App Engine \(標準環境\) 遷移指南](#)
- [App Engine 定價和配額資訊](#)
- [推出第二代 App Engine 平台 \(2018 年\)](#)

- [比較第一代和第二代平台](#)
- [對舊版執行階段的長期支援](#)
- [說明文件遷移範例](#)
- [社群提供的遷移範例](#)

其他雲端資訊

- [在 Google Cloud Platform 上執行 Python](#)
- [Google Cloud Python 用戶端程式庫](#)
- [Google Cloud「永久免費」方案](#)
- [Google Cloud SDK](#) (`gcloud` 指令列工具)
- [所有 Google Cloud 說明文件](#)

影片

- [無伺服器遷移工作站](#)
- [無伺服器 Expeditions](#)
- 訂閱 [Google Cloud Tech](#)
- 訂閱 [Google Developers](#)

授權

這項內容採用的授權為 [Creative Commons 姓名標示 2.0 通用授權](#)。
